KING SAUD UNIVERSITY
COLLEGE OF COMPUTER AND INFORMATION SCIENCES
DEPARTMENT OF COMPUTER SCIENCE

جامعة الملك سعود
King Saud University
1957
كلية علوم الحاسب والمعلومات
قسم هندسة البرمجيات

Project Report

Course Code / Title: CSC212 – Data Structures
Assessment: Project
Semester / Year: Fall 2024 **Submission Date**: 23/11/2024

Simple Search Engine

Instructor: Dr. Nasser Al-Sadhan

Section: 88358

STUDENT IDENTIFICATION:

Name: Mohammed Abuhaimed **ID**: 444100955
Name: Mohammed Alamri **ID**: 439101248
Name: Osamah AlOsaimi **ID**: 443100281

Table of Contents

Work Distribution:

Mohammed Abuhaimed: *Leader + Implementation + GitHub Administrator*

Osamah AlOsaimi: *Implementation + Performance Analysis*

Mohammed Alamri: *Documentation + Class Diagram*

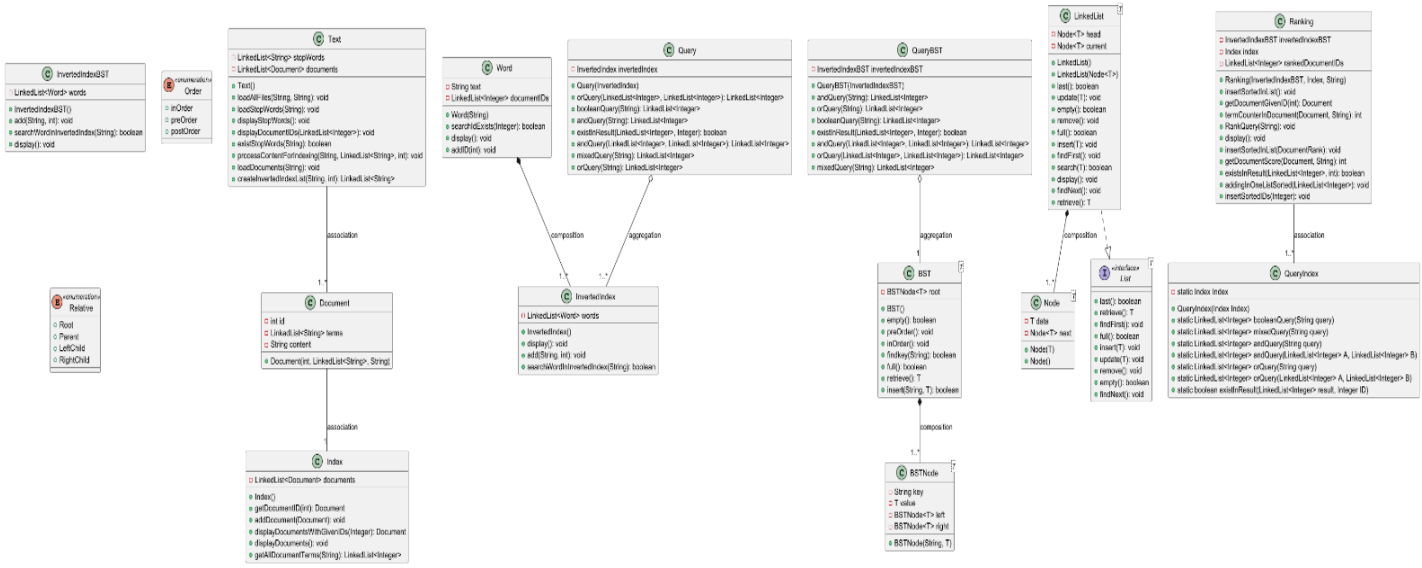
GitHub Repository: **https://github.com/MohammedHAbuhaimed/CSC-212**

1. Introduction

1.1 Overview: Simple Search Engine

The Simple Search Engine represents a software system for efficient storing, management, and retrieval of textual data. In its essence, the system has the objective of enabling the user to search terms or phrases within a set of indexed documents and returns relevant results with regards to queries provided. Advanced data structures comprising Binary Search Trees, linked lists, and inverted indexes are utilized in this project to assure maximum efficiency in both indexing and query operations.

2. Class Diagram:



**Due to the class diagram size, it may not be clear, Please find it attached in the GitHub repositiry.*

3. Performance Analysis

3.1 Big-O Time Complexity Analysis:

Class BST<T>:

Method	Description	Big-O
add (T element)	Add an element to the BST	O(log n)
remove (T element	Remove an element from BST	O(log n)
search (T element)	Search for an element	O(log n)

Class Document:

Method	Description	Big-O
getId()	Rederive document's ID	O(1)

Class Index:

Method	Description	Big-O
addDocument(Document d)	Add a document to index	O(1)
displayDocuemnts()	Displays documents	O(n)
GetAllDocGivenID(int id)	Return document with the id searched for	O(n)

Class InvertedIndex:

Method	Description	Big-O
AddWord(Word w)	Add a word to index	O(1)
searchWord(String w)	Search a word in the index	O(n)

Class InvertedIndexBST:

Method	Description	Big-O
AddWord(Word w)	Add a word to BST	O(log n)
searchWord(String w)	Search a word in BST	O(log n)

Class LinkedList<T>:

Method	Description	Big-O
insert(T e)	Inserts a new element	O(1)
search(T x)	Search an element	O(n)
display()	Print all elements	O(n)

Class Query:

Method	Description	Big-O
booleanQuery(String query)	Process a boolean query	O(n)
mixedQuery(String query)	Merge result with AND operation	O(n)

Class QueryIndex:

Method	Description	Big-O
mixedQuery(String query)	Merge result with AND operation	O(n)

Class QueryBST

Method	Description	Big-O
mixedQuery(String query)	Merge result with AND operation	O(n)

Class Ranking:

Method	Description	Big-O
Doc_Rank(int id, int rank)	initializes a document rank object with its ID and rank	O(1)
Ranking(InvertedIndexBST inverted, index index1, String Query)	Initializes the `Ranking` object with an inverted index, main index, and query.	O(1)
displayAllDocList()	Displays all ranked documents and their scores	O(n)
termFrequency(Document document, String term)	Counts occurrences of a term in a document with `w` words.	O(n)

Class Text:

Method	Description	Big-O
LoadStopWords(String fileName)	Reads stop words from a file and saves them.	O(n)
LoadAllDoc(String fileName)	Reads documents from a file and laod it	O(n)
makeLinkedListOfWords(String content, int id)	Converts document content into a linked list of words and updates indices.	O(n)

Class Word:

Method	Description	Big-O
addID(int id)	Add a docuemnt ID	O(1)
searchIdExists(int id)	Search for a document ID	O(1)

3.2 Big-O Time Comparison:

Index : Relatively faster for small datasets because of its simpler structure but scales poorly with bigger sets due to linear search.

Inverted Index: More efficient for retrieval when datasets get bigger because terms are pre-mapped to document IDs. However, looking up terms in the linked list is, which may get inefficient.

Inverted Index with BST: Most efficient for large datasets, because the BST supports faster searches of terms. It balances insertion and retrieval effectively; hence, it is the best to be used for scalable Boolean retrieval.

3.3 Classes Overview

3.3.1. BST (Binary Search Tree)

Purpose: Implements a binary search tree (BST) data structure to store and retrieve elements efficiently.

Responsibilities:

Organizes elements in a hierarchical tree structure.

Provides efficient insertion, deletion, and search operations.

3.3.2. Document

Purpose: Represents a document in the system.

Responsibilities:

Stores information about the document, such as its ID and content.

Acts as a fundamental unit for indexing and retrieval in the system.

3.3.3. Index

Purpose: Serves as an abstract base class for indexing strategies.

Responsibilities:

Provides a common interface for different indexing techniques (e.g., inverted index, BST-based index).

Defines methods for adding, removing, and searching for words or documents.

3.3.4. InvertedIndex

Purpose: Implements an inverted index for efficient document retrieval.

Responsibilities:

Maps words to the documents in which they appear.

Supports fast lookups to identify relevant documents for a given query.

3.3.5. InvertedIndexBST

Purpose: Combines an inverted index with a BST structure.

Responsibilities:

Uses a binary search tree to manage the inverted index data.

Provides hierarchical organization for words and associated documents.

3.3.6. LinkedList

Purpose: Implements a linked list data structure.

Responsibilities:

Stores a sequence of elements in nodes, with each node pointing to the next.

Provides operations for adding, removing, and traversing elements.

3.3.7. Query

Purpose: Represents a search query in the system.

Responsibilities:

Stores query terms and parameters.

Acts as input for searching through the index.

3.3.8. QueryBST

Purpose: Handles queries using a BST-based approach.

Responsibilities:

Optimizes query processing by leveraging the hierarchical organization of a BST.

Searches for terms and retrieves relevant documents.

3.3.9. QueryIndex

Purpose: Processes queries using an index.

Responsibilities:

Interfaces with the index to execute search operations.

Retrieves and ranks documents relevant to the query.

3.3.10. Ranking

Purpose: Implements ranking algorithms to prioritize search results.

Responsibilities:

Scores documents based on relevance to the query.

Orders search results by their scores.

3.3.11. Text

Purpose: Represents text data for processing.

Responsibilities:

Handles operations related to text parsing and manipulation.

Prepares text for indexing or querying (e.g., tokenization, normalization).

3.3.12. Word

Purpose: Represents an individual word in the system.

Responsibilities:

Stores the word itself and its associated metadata (e.g., frequency, position).

Acts as a fundamental unit for indexing and querying.